# Task 3 Part A: FX rates collection

**API used:** open.er-api.com (`/v6/latest/{base}`).

| File | What it is |
|---|---|
| `FX_Rates_open-er-api.postman_collection.json` | The collection |
| `FX_Rates_open-er-api.postman_environment.json` | Environment: base URL, response-time limit, tolerance and the negative test codes |
| `fx_base_currencies.json` | Data file: 4 base currencies (USD, EUR, GBP, AED) |
| `newman_run_output.txt` | Last run: 4 iterations, 10 requests, 36 checks, 0 failed |

**How to run:** in the Postman Collection Runner, choose the environment, then select `fx_base_currencies.json` as the iteration data.

## What the assertions verify, and why

| Assertion | Why |
|---|---|
| Status is 200 | The call worked. |
| Response time under 2 seconds | The customer waits on the transfer screen while the rate loads. |
| Response matches the schema: result, base_code, update time, and rates as numbers | If the shape changes, the app can't show a rate. |
| base_code is the requested currency | Proves the rates are for the currency we asked for. The USDX case below shows why this matters. |
| Expected target currencies are present, with positive rates | The currencies customers send money to must be there. A missing or zero rate would show the wrong amount. |
| One rate is in a sensible range (set per data row) | Catches wrong values, not just a wrong format. AED is fixed to USD, so its range is tight. |

## Captured value

Request 1 captures the rate from the base to the check currency (for example USD to EUR). Request 2 gets the rates for the check currency, and checks that the reverse rate is 1 divided by the captured rate, within 0.5%.

**Why this value:** it's the number the customer sees. If the two directions don't match, the provider is inconsistent, and the app could quote different rates for the same pair.

## Data file and variables

- **Data file:** each row has a base currency, the target currencies it must include, a check currency and its sensible range. The same 2 requests run once per row.
- **Environment variables:** the base URL, time limit, tolerance and negative test codes. Nothing is hard-coded in the requests.

## Negative cases

| Case | What the API actually does | What I assert | OK for a banking app? |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `XYZ`: unsupported code | HTTP 200, with `result: "error"` and `error-type: "unsupported-code"` | Status 200, the error fields, and no rates | No. An error with a 200 status looks like success to monitoring. The app must check `result`, and a 4xx would be better. |
| `USDX`: malformed, 4 letters | HTTP 200, `"success"`, and rates for USD. The code is cut to 3 letters. | `base_code` is `USD`, not `USDX` | No. A typo returns a real-looking rate for a different currency. The app must check that `base_code` matches the request. |